

ShopFlow : optimiser une base PostgreSQL

LES CHIFFRES

Ce que j'ai fait	
Atelier 1	Diagnostic des requêtes
Atelier 2	Réécriture de requêtes
Atelier 3	Index de l'historique client
Atelier 4	Index spécialisés
La base	1 000 clients, 200 produits, 100 000 commandes, 300 000 lignes
Moteur	PostgreSQL 18.6

À LIRE

Bonjour. Mon travail porte sur ShopFlow, une boutique en ligne simulée sous PostgreSQL, avec 100 000 commandes. L'objectif du cours : optimiser une base en diagnostiquant d'abord, puis en agissant, et en prouvant que chaque changement marche. J'ai fait quatre ateliers : un diagnostic, une réécriture de requêtes, puis deux ateliers d'index.

Ma méthode : mesurer sans se tromper

LES CHIFFRES

Règle	En pratique
1	Résultat vérifié avant la vitesse (lignes, totaux, empreinte)
2	3 échauffements, puis 5 mesures, médiane
3	Mesures alternées (51 tours) et valeurs stables : pages lues, journal d'écriture (WAL), tailles
4	Laboratoire remis à l'état initial, vérifié à chaque fois
Constat	La même requête a pris 0,23 ms puis 0,55 ms : les durées sont bruitées

À LIRE

Ma méthode tient en quatre règles. Un : je vérifie que le résultat est identique avant de regarder la vitesse. Deux : je répète les mesures et je prends la médiane. Trois : comme les durées d'une milliseconde varient d'une exécution à l'autre, je m'appuie surtout sur des valeurs stables : les pages lues, le volume du journal d'écriture et la taille des index. Quatre : je remets le laboratoire dans son état initial après chaque expérience. En route, j'ai aussi corrigé deux défauts dans mes propres mesures.

Atelier 1 : le diagnostic

LES CHIFFRES

Requête	Résultat vérifié	Médiane
Historique du client 42	20 lignes, 14 400,00 €	0,229 ms
Agrégation mensuelle	30 groupes, 29 775 001,25 €	84,1 ms
Nœud coûteux	Tri sur disque, 48 % du temps	
À froid	1 096,8 ms (13 fois plus lent)	
Essai : plus de mémoire (copie)	84,6 → 50,2 ms	×1,7
Essai : statistique (copie)	84,6 → 45,5 ms	×1,9

À LIRE

Atelier un, le diagnostic. Deux requêtes : l'historique d'un client et une agrégation mensuelle. L'agrégation prend 84 millisecondes, et son coût principal est un tri de 100 000 lignes qui déborde sur le disque, car la mémoire de tri n'est que de 4 mégaoctets. PostgreSQL se trompe aussi : il prévoit 80 000 groupes au lieu de 30. Sur une copie, plus de mémoire rend la requête 1,7 fois plus rapide, et une statistique 1,9 fois plus rapide, ce qui corrige la cause. À froid, elle était 13 fois plus lente.

Atelier 2 : réécrire sans changer le résultat

LES CHIFFRES

Variante	Lignes	Médiane	Verdict
Jointure	80 000 (800 clients)	27,6 ms	Rejetée : doublons
Jointure + DISTINCT	800	40,2 ms	Correcte mais lente
EXISTS	800	11,3 ms	Retenue : ×2,4
Total par commande, faux	143 280 €	1,10 ms	Rejetée : ×3
Total par commande, juste	47 760 €	2,05 ms	Retenue

À LIRE

Atelier deux, réécrire sans changer le résultat. Pour lister les clients qui ont une commande payée, une jointure renvoie 80 000 lignes pour 800 clients : le client 42 apparaît 100 fois. DISTINCT cache le problème mais reste lent. La version EXISTS donne les mêmes 800 clients en 11 millisecondes, 2,4 fois plus vite. Deuxième cas : additionner le total des commandes après une jointure avec leurs lignes triple le montant, 143 280 euros au lieu de 47 760. Cette version est presque deux fois plus rapide, mais fausse, donc je la rejette.

Atelier 3 : quel index pour l'historique client

LES CHIFFRES

Variante	Médiane	Pages lues	Écriture (WAL)	Décision
Sans index adapté	0,690 ms	96	référence	
Index simple	0,729 ms	96	+28,2 %	Rejeté : aucun gain
Index composé	0,274 ms (×2,5)	27	+35,0 %	Retenu
Index couvrant	0,199 ms (×3,5)	7	+42,1 %	Option
Client à 20 100 commandes	23,3 → 0,26 ms	287 → 27		×91

À LIRE

Atelier trois, l'historique d'un client, mesuré sur la requête exacte de l'API. L'index simple sur le client n'apporte rien, car la base en avait déjà un qui commence par la même colonne. L'index composé, client puis date, supprime le tri : 2,5 fois plus rapide. Pour un client très actif de 20 100 commandes, on passe de 23 millisecondes à 0,26, soit 91 fois. Il coûte 35 % de journal d'écriture en plus, et je le retiens. L'index couvrant va un peu plus vite mais il est 45 % plus gros : option. Un piège : si l'ORDER BY n'est pas préfixé du nom de la table, le tri porte sur du texte et l'index devient inutile.

Atelier 4 : un index spécialisé par besoin

LES CHIFFRES

Besoin	Index	Résultat mesuré	Décision
Commandes en attente (10 %)	Partiel	10,1 → 0,07 ms (×145), index 12 fois plus petit, WAL +3,2 %	Retenu
Catalogue, 200 produits	GIN	Jamais utilisé : 3 pages à lire	Rejeté
Catalogue, 200 000 lignes	GIN	×17 (valeur rare), ×2,0 (25 %) ; WAL +120,7 %	Conditionnel
Propriété connue	Expression ou colonne	×28 et ×30, WAL +35 à +39 %	Préférable
Réservations	GiST	27,5 → 0,168 ms (×164), insertions ×7,8 plus lentes	Si besoin

À LIRE

Atelier quatre, un index par besoin. Pour la file des commandes en attente, soit 10 % de la table, un index partiel passe de 10 millisecondes à 0,07, 145 fois plus vite, avec un index 12 fois plus petit qu'un index complet. Limite : il ne sert que ce statut, avec la valeur écrite en clair. Pour le catalogue JSON, le GIN n'est jamais utilisé sur nos 200 produits, donc je le rejette pour l'instant. À 200 000 lignes il gagne 17 fois sur une valeur rare, mais un index sur la seule catégorie fait mieux, 28 à 30 fois, et coûte trois fois moins d'écriture. Enfin, le GiST trouve les chevauchements de périodes 164 fois plus vite, mais ralentit les insertions.

Ce que je retiens

LES CHIFFRES

Idée	
1	On diagnostique avant de modifier
2	Une optimisation se juge sur le gain, le coût d'écriture et le résultat inchangé
3	On choisit l'index d'après la requête et son opérateur
4	Un index sans gain est un résultat à expliquer, pas à cacher
5	Je vérifie mes propres mesures
Limites	Données synthétiques, durées variables, migrations testées mais non appliquées
Suite	Atelier 4 bis : migration de schéma

À LIRE

Pour conclure, cinq idées. Un : on diagnostique avant de modifier. Deux : une optimisation se juge sur le gain, le coût d'écriture et le résultat inchangé. Trois : on choisit l'index d'après la requête et son opérateur. Quatre : un index sans gain, comme l'index simple ou le GIN sur 200 lignes, est un résultat à expliquer, pas à cacher. Cinq : je vérifie mes propres mesures, et j'ai corrigé deux défauts avant de conclure. Limites : données synthétiques et durées variables. Prochaine étape : la migration de schéma. Merci.